Лабораторная работа 0. Воспроизводимый эксперимент: подбор гиперпараметров нейросети

Главный вопрос. Дал ли подбор гиперпараметров улучшение, которое больше случайного разброса?
Что получится. Каркас, который одной командой обучает и оценивает небольшую нейросеть, сохраняет всё для повторения запуска и записывает результаты в MLflow; поиск гиперпараметров в Optuna; отчёт с честным сравнением.
Среда. Python 3.10+, PyTorch, scikit-learn, Optuna 5, MLflow 3; достаточно CPU — личный компьютер, Colab или Kaggle.
Дальше пригодится. Каркас и журнал MLflow используются во всех следующих работах. Общие требования к ним — в «Правилах», п. 4.

1. Зачем эта работа

Коллега сообщает: «Я подобрал гиперпараметры, точность выросла с 95,1 до 96,3%». Можно ли этому верить? За таким числом может стоять не только лучшая настройка:

- сеть обучалась с другим seed, и ей повезло с начальными весами;
- второй замер сделан на другом разбиении данных;
- перебрали 50 вариантов и показали лучший — на той же выборке, по которой выбирали;
- код между запусками менялся, и никто не помнит как.

Каждая из этих причин даёт «плюс процент» без настоящего улучшения. Чтобы ответить коллеге, нужны сохранённые условия каждого запуска и способ честно сравнивать запуски между собой.

В этой работе вы построите **экспериментальный каркас** (experiment harness) — программу, которая запускает обучение по файлу конфигурации, сохраняет всё, что нужно для повторения, и записывает результаты в журнал. Затем с его помощью вы сделаете то же, что коллега, — подберёте гиперпараметры в Optuna — и проверите, больше ли найденное улучшение случайного разброса.

Объект опытов — небольшая полносвязная нейросеть (MLP), которая распознаёт рукописные цифры на изображениях 8×8 пикселей. Она обучается за секунды на CPU, но в ней уже есть всё, что делает эксперимент с настоящей моделью трудным: случайная инициализация и порядок примеров, гиперпараметры, неудачные запуски и соблазн выбрать лучший результат. В следующих работах тот же каркас будет запускать языковую модель, поиск и агентов: объект меняется, правила остаются.

2. Как устроено

2.1. Конфигурация, запуск, серия

- **Конфигурация** — файл с параметрами: гиперпараметры, seed, пути к данным. Варианты опыта различаются только конфигурацией; код между ними не меняется.
- **Запуск** — одно обучение и оценка по одной конфигурации, одна команда, например `python train.py --config configs/base.yaml`. У каждого запуска свой `run_id` и каталог `runs/<run_id>/`.
- **Серия** — заранее перечисленный набор запусков: например, базовая конфигурация с пятью разными seed.
- **Эксперимент** — сравнение серий, отвечающее на вопрос.

Рис. 1 показывает, как эти части связаны. Поиск в Optuna — тоже серия: каждый его шаг (trial) вызывает ту же функцию обучения, что и обычный запуск.

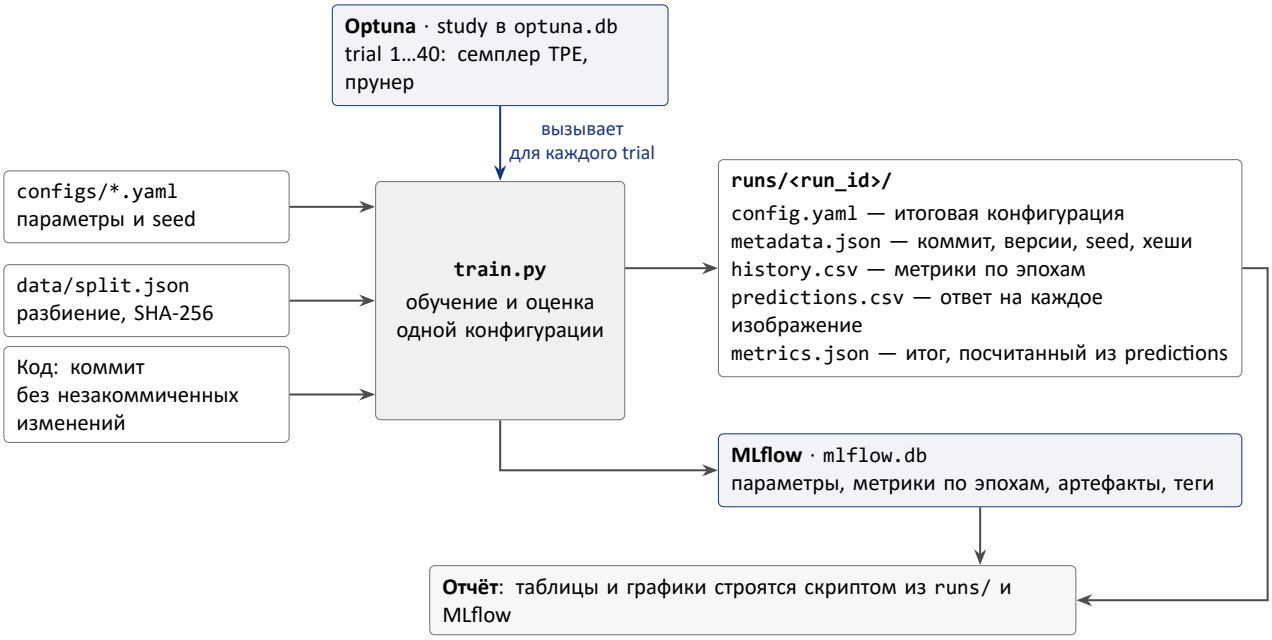


Рис. 1. Каркас ЛР0: входы запуска, каталог запуска, журнал MLflow и поиск Optuna

2.2. Откуда берётся случайность

При обучении случайны начальные веса, порядок мини-батчей и маски dropout. Всё это выдаёт генератор псевдослучайных чисел, а **seed** задаёт его начальное состояние. Тот же seed при тех же коде, данных и окружении даёт те же числа: на CPU — вплоть до последнего бита, на GPU часть операций недетерминирована [2]. Другой seed даёт другую сеть и немного другую точность. Этот разброс — естественный шум процедуры обучения, и эффект от подбора нужно сравнивать именно с ним [3].

Разбиение данных задаётся отдельным параметром `split_seed` и одинаково во всех запусках. Иначе запуски сравнивались бы на разных данных.

2.3. Обучение, валидация, тест

Набор делится на три части:

- **train** — на ней обучаются веса;
- **val** — по ней сравниваются конфигурации и выбирается лучшая;
- **test** — на ней выбранная конфигурация оценивается один раз, в самом конце.

Третья часть нужна вот зачем. Если выбрать лучшую из 40 конфигураций по точности на val, эта точность оптимистична: часть преимущества — удача на конкретных 360 изображениях. Это **смещение отбора** (selection bias) [4]. Тест в выборе не участвовал, поэтому его оценка честная, но только пока на него смотрят один раз.

Для масштаба: одно изображение из 360 — это 0,28 п.п., разница в 1 п.п. — примерно 3–4 изображения.

2.4. Поиск гиперпараметров в Optuna

Optuna [5] организует поиск как **study** — последовательность **trial**. В каждом trial семплер предлагает значения гиперпараметров, а функция цели обучает сеть и возвращает точность на val. Семплер по умолчанию, TPE, использует результаты прошлых trial, чтобы чаще предлагать значения из удачных областей [6]. **Прунер** останавливает бесперспективные trial досрочно: после каждой эпохи trial сообщает текущую точность, и `MedianPruner` прерывает его, если она хуже медианы прошлых trial на той же эпохе.

У trial три итоговых состояния: `COMPLETE`, `PRUNED` (остановлен прунером) и `FAIL`. Если функция цели вернула NaN, trial получает `FAIL`, а поиск продолжается. Если она выбросила исключение, поиск по умолчанию прерывается; чтобы продолжить, тип исключения перечисляют в параметре `catch` [7]. Так в Optuna выглядит общее правило курса: отказ одной записи сохраняется и не останавливает серию, а неожиданная ошибка останавливает её с понятной причиной.

Study хранится в базе SQLite, поэтому поиск, прерванный вместе с сессией Colab, продолжается с того же места.

У семплера тоже есть seed. `TPESampler(seed=42)` при последовательном выполнении повторяет ту же последовательность предложений. При параллельном выполнении (`n_jobs > 1`) Optuna меняет seed семплеров, и последовательность уже не повторяется [7]. Параллельность ускоряет поиск ценой повторяемости, поэтому в этой работе поиск последовательный.

2.5. Журнал MLflow

MLflow Tracking [8] хранит запуски: параметры, метрики — в том числе по эпохам, файлы-артефакты и теги. Хранилище по умолчанию — база `mlflow.db` в текущем каталоге. Каждый запуск каркаса и каждый trial — отдельная запись MLflow; trial одного поиска вкладываются в общую родительскую запись.

3. Что сделать

3.1. Данные и разбиение

1. Загрузите `load_digits()` из scikit-learn [9]: 1797 изображений 8×8 (64 признака со значениями от 0 до 16) и метки от 0 до 9. Скачивать ничего не нужно.
2. Разделите данные со стратификацией по классам: сначала отделите test (20%), затем от остатка — val (25% остатка). Получится 1077, 360 и 360 изображений. Используйте `split_seed = 0`; код — в приложении.
3. Сохраните индексы частей в `data/split.json`, вычислите SHA-256 файла и запишите в манифест данных источник (scikit-learn и его версия), правило разбиения, размеры частей и хеш («Правила», п. 5).
4. Проверьте, что части не пересекаются, в сумме дают 1797 и в каждой есть все 10 классов.

Признаки делятся на 16. Масштаб известен заранее, статистики по данным не вычисляются, и сведения о val и test в обучение не попадают.

3.2. Модель и запуск

Сеть: 64 входа → `n_layers` скрытых слоёв по `hidden` нейронов с ReLU и dropout → 10 выходов. Функция потерь — кросс-энтропия, оптимизатор — Adam, число эпох фиксировано: 30. Если вы раньше не обучали нейросети в PyTorch, пройдите первые шесть разделов учебника «Learn the Basics» [1] — этого достаточно для работы. Пример базовой конфигурации:

```yaml
experiment: lr0
split_seed: 0
seed: 0
model: {n_layers: 1, hidden: 64, dropout: 0.0}
train: {lr: 0.001, weight_decay: 0.0, batch_size: 64, epochs: 30}
runtime: {device: cpu, num_threads: 1}
```

Команда запуска: `python train.py --config configs/base.yaml [--seed N] [--eval-test]`. Аргументы командной строки переопределяют значения из файла, а в каталог запуска сохраняется **итоговая** конфигурация. Оценка на test выполняется только с флагом `--eval-test`.

Чтобы запуск с тем же seed повторялся, задайте seed для `random`, NumPy и PyTorch, передайте в `DataLoader` генератор с тем же seed, используйте `num_workers=0`, включите `torch.use_deterministic_algorithms(True)` и зафиксируйте число потоков через `torch.set_num_threads` [2].

3.3. Каталог запуска и журнал

Каталог `runs/<run_id>/` устроен по «Правилам» (п. 4.2); для обучения нейросети файлы такие:

Файл	Что в нём
`config.yaml`	Итоговая конфигурация
`metadata.json`	`run_id`, серия, статус, время начала и конца, коммит, версии Python, PyTorch, scikit-learn, Optuna и MLflow, число потоков, seed, SHA-256 файла `split.json`
`history.csv`	По эпохам: `epoch`, `train_loss`, `val_loss`, `val_acc`, время эпохи
`predictions.csv`	По изображениям val (и test при `--eval-test`): `split`, `index`, `y_true`, `y_pred`
`metrics.json`	Итоговые `val_acc`, `val_loss`, время обучения и `test_acc` при `--eval-test`; всё вычисляется из `predictions.csv` и `history.csv`

Строки `history.csv` дописываются после каждой эпохи. Статус запуска — `running`, затем `completed` или `failed`; при ошибке сохраняются текст исключения с трейсбеком (traceback) и уже записанные файлы. Итоговый запуск не начинается из незакоммиченного кода («Правила», п. 4.3); каталоги `runs/` и `mlartifacts/`, файлы `mlflow.db` и `optuna.db` внесите в `.gitignore`.

В MLflow каждый запуск записывает параметры конфигурации, метрики по эпохам (`step` — номер эпохи), итоговые метрики, файлы каталога как артефакты и теги `commit`, `series`, `seed`.

3.4. Проверки перед опытами

Это тесты реализации; их результаты в таблицы отчёта не входят.

- Точность считается верно: предсказания `[1, 2, 3, 3]` при ответах `[1, 2, 3, 4]` дают 0,75.
- Точность, пересчитанная из `predictions.csv`, совпадает с числом, полученным при обучении.
- Два запуска с seed 0 дают одинаковые `history.csv` и `predictions.csv`, запуск с seed 1 — другие.
- Если файл кода изменён и не закоммичен, итоговый запуск отказывается начинаться.
- Конфигурация с `lr: -1` отклоняется до начала обучения с понятным сообщением.

4. Эксперименты

До запусков запишите прогноз («Правила», п. 6): насколько велик будет разброс точности на val по seed; насколько лучший trial Optuna превзойдёт базовую конфигурацию на val; сохранится ли улучшение при переобучении с новыми seed и на test. Закоммитьте прогноз вместе с конфигурациями.

4.1. Эксперимент 1. Повторяемость и разброс

1. Запустите базовую конфигурацию дважды с seed 0. Файлы `history.csv` и `predictions.csv` должны совпасть полностью; если это не так, найдите причину, прежде чем двигаться дальше.
2. Серия `base_seeds`: базовая конфигурация с seed от 1 до 5. Для каждого запуска приведите точность на val и время обучения, в сводке — минимум, медиану и максимум точности.
3. Сравните предсказания двух запусков с разными seed: сколько изображений val первый классифицировал верно, а второй — нет, и наоборот. Близкая точность может скрывать разные ошибки.

4.2. Эксперимент 2. Поиск гиперпараметров

Пространство поиска:

Параметр	Значения
`lr`	от 1e-4 до 1e-1, логарифмическая шкала
`n_layers`	1, 2 или 3

Параметр	Значения
`hidden`	от 32 до 512, целое, логарифмическая шкала
`dropout`	от 0 до 0,5
`weight_decay`	от 1e-6 до 1e-2, логарифмическая шкала
`batch_size`	32, 64 или 128

Число эпох (30), разбиение и seed обучения (0) одинаковы для всех trial. Функция цели возвращает точность на val после последней эпохи, а после каждой эпохи сообщает прунеру текущую точность.

1. Создайте study `lr0_search` в `sqlite:///optuna.db` с `TPESampler(seed=42)` и `MedianPruner(n_startup_trials=5, n_warmup_steps=5)`. Выполните 40 trial последовательно. Каждый trial — вложенная запись MLflow с номером trial и итоговым состоянием.
2. Прервите поиск примерно после 10 trial и запустите снова с `load_if_exists=True`. Нумерация trial должна продолжиться, а общее число trial — дойти до 40.
3. Для отчёта: число trial в каждом состоянии; сколько эпох реально выполнено из 40 × 30 = 1200 и сколько времени сэкономил прунер; график «номер trial — точность на val» с линией лучшего значения на данный момент; важность параметров по `optuna.importance.get_param_importances`. Графики стройте из `study.trials_dataframe()` или средствами `optuna.visualization`.

4.3. Эксперимент 3. Честное сравнение

1. Сохраните параметры лучшего trial в `configs/best.yaml`.
2. Серия `final`: базовая и лучшая конфигурации, каждая с seed от 101 до 105 — эти seed в поиске не использовались, — с флагом `--eval-test`. Всего 10 запусков.
3. Для каждого seed s вычислите разность точностей на val и на test:

$$\Delta_s = \mathrm{acc}_{\mathrm{best}}(s) - \mathrm{acc}_{\mathrm{base}}(s).$$

   Покажите все пять значений, их медиану и диапазон, а также сколько изображений test лучшая конфигурация исправила и сколько испортила — суммарно по пяти seed.
4. Сравните три числа: точность лучшего trial в поиске, среднюю точность лучшей конфигурации на val с новыми seed и её среднюю точность на test. Объясните, куда делась часть улучшения.
5. Ответьте на главный вопрос работы, сопоставив Δ_s с разбросом из эксперимента 1. Если все пять разностей на test положительны и больше этого разброса, улучшение устойчиво. Если разности разного знака, данных недостаточно, чтобы говорить об улучшении.

Test используется только здесь. Если после него вы что-то изменили в системе, он перестал быть отложенным, и об этом нужно сказать в отчёте.

4.4. По желанию

- Повторите поиск с другим seed семплера. Найдётся ли та же лучшая конфигурация и с той же ли точностью?
- Сравните TPE со случайным поиском (`RandomSampler`) при том же бюджете в 40 trial.
- Запустите поиск дважды с `n_jobs=2` и одинаковым seed семплера. Совпадут ли последовательности предложенных параметров?

5. Что сдать

- **Репозиторий**: код, `configs/` (базовая, лучшая конфигурации и пространство поиска), `data/split.json`, манифест данных, прогноз, отчёт и инструкция запуска. В `.gitignore` — каталоги запусков и базы.
- **Артефакты**: каталоги запусков экспериментов 1 и 3, `optuna.db`, `mlflow.db` с каталогом артефактов или выгрузка `mlflow.search_runs()` в CSV.

- **Отчёт** на 2–3 страницы («Правила», п. 8): прогноз и результат; таблица эксперимента 1; итоги поиска с двумя графиками; таблица разностей Δ_s и вывод; что сгенерировано и как проверено.

Ошибки постановки («Правила», п. 10), которые чаще всего встречаются в этой работе: выбор конфигурации по test; точность лучшего trial выдаётся за итоговый результат; trial в состояниях `FAIL` и `PRUNED` исчезают из отчёта.

Вопросы к защите

1. Два запуска с одинаковым seed дали разную точность. Что вы проверите в первую очередь?
2. Почему точность лучшего trial на val оптимистична? Покажите это на своих числах.
3. Разница конфигураций на test — 0,8 п.п. Сколько это изображений и как это соотносится с разбросом по seed?
4. По журналу MLflow восстановите параметры лучшего trial и повторите его одной командой. Совпал ли результат с записанным и почему должен совпасть?
5. Что изменится при `n_jobs=4`: что вы выиграете и что потеряете?
6. Увеличьте `n_warmup_steps` прунера до 15. Предскажите, как изменятся число выполненных эпох и время поиска, затем проверьте на 10 trial.
7. Почему точность в `metrics.json` пересчитывается из `predictions.csv`, а не записывается из памяти?
8. Зачем разбиению отдельный `split_seed`? Что сломается, если разбивать данные тем же seed, что и обучать?

Источники

1. PyTorch. Learn the Basics — тензоры, датасеты, построение сети и цикл обучения; для этой работы достаточно первых шести разделов.
2. PyTorch. Reproducibility — seed, `use_deterministic_algorithms`, `DataLoader` и ограничения на GPU.
3. Bouthillier X. et al. Accounting for Variance in Machine Learning Benchmarks. MLSys, 2021. Статья — насколько велик разброс от seed и порядка данных и как учитывать его при сравнении (разд. 2–3).
4. Cawley G. C., Talbot N. L. C. On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. JMLR, 11, 2010. Статья — почему оценка выбранной модели по той же выборке смещена (разд. 1–2).
5. Akiba T. et al. Optuna: A Next-generation Hyperparameter Optimization Framework. KDD, 2019. Статья. Документация: эффективные алгоритмы и прунинг, MedianPruner.
6. Bergstra J. et al. Algorithms for Hyper-Parameter Optimization. NeurIPS, 2011. Статья — исходное описание TPE (разд. 4).
7. Optuna. FAQ: воспроизводимые результаты, исключения в trial, NaN в trial.
8. MLflow. ML Experiment Tracking — параметры, метрики, артефакты, вложенные запуски.
9. scikit-learn. load_digits.

Приложение. Фрагменты для реализации

Разбиение (даёт 1077 / 360 / 360):

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

X, y = load_digits(return_X_y=True)
idx = np.arange(len(y))
rest, test = train_test_split(idx, test_size=0.2, stratify=y, random_state=0)
train, val = train_test_split(rest, test_size=0.25, stratify=y[rest], random_state=0)
```

Проверка чистого рабочего дерева перед итоговым запуском:

```
import subprocess

dirty = subprocess.run(["git", "status", "--porcelain"], capture_output=True,
 ↪  text=True).stdout.strip()
if dirty:
    raise SystemExit("Есть незакоммиченные изменения — итоговый запуск невозможен:\n" + dirty)
commit = subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
 ↪  text=True).stdout.strip()
```

Поиск с продолжением после обрыва. Функция `train_epochs` — ваша функция обучения из `train.py`, которая после каждой эпохи возвращает номер эпохи и точность на val.

```
import math

import mlflow
import optuna
from optuna.study import MaxTrialsCallback

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("lr0")

def objective(trial):
    params = {
        "lr": trial.suggest_float("lr", 1e-4, 1e-1, log=True),
        "n_layers": trial.suggest_int("n_layers", 1, 3),
        "hidden": trial.suggest_int("hidden", 32, 512, log=True),
        "dropout": trial.suggest_float("dropout", 0.0, 0.5),
        "weight_decay": trial.suggest_float("weight_decay", 1e-6, 1e-2, log=True),
        "batch_size": trial.suggest_categorical("batch_size", [32, 64, 128]),
    }
    with mlflow.start_run(run_name=f"trial-{trial.number}", nested=True):
        mlflow.log_params(params)
        for epoch, val_acc in train_epochs(params, seed=0, epochs=30):
            mlflow.log_metric("val_acc", val_acc, step=epoch)
            trial.report(val_acc, step=epoch)
            if trial.should_prune():
                mlflow.set_tag("state", "PRUNED")
                raise optuna.TrialPruned()
        mlflow.set_tag("state", "FAIL" if math.isnan(val_acc) else "COMPLETE")
        return val_acc

study = optuna.create_study(
    study_name="lr0_search", storage="sqlite:///optuna.db", load_if_exists=True,
    direction="maximize", sampler=optuna.samplers.TPESampler(seed=42),
    pruner=optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=5),
)
with mlflow.start_run(run_name="lr0_search"):
    study.optimize(objective, n_trials=40, callbacks=[MaxTrialsCallback(40, states=None)])
```

`MaxTrialsCallback(40, states=None)` останавливает поиск, когда в study набралось 40 trial в любых состояниях, поэтому повторный запуск после обрыва выполняет только недостающие. При продолжении seed семплера не восстанавливает прежнюю последовательность: повторяемым считается только поиск, выполненный без перерыва.

Ориентиры для самопроверки (CPU, один поток, PyTorch 2, Optuna 5). Один запуск базовой конфигурации занимает 1–3 секунды. Точность базовой конфигурации на val при разных seed — около 95–96%, разброс по пяти seed — меньше 1 п.п. Поиск из 40 trial идёт 2–3 минуты, прунер останавливает примерно половину trial и экономит 30–50% эпох. Если ваши числа отличаются в разы, проверьте разбиение, масштаб признаков и границы таймера.